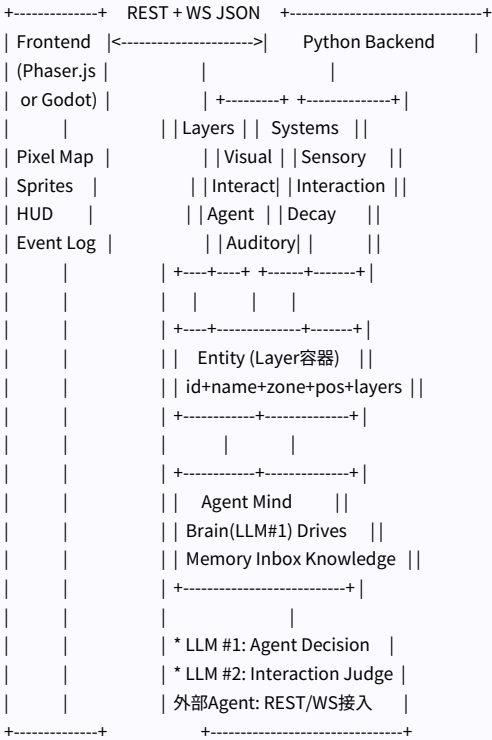


AgentWorld Async — 完整技术架构

1. 系统全景

AgentWorld Async 是一个异步、分层架构、LLM 驱动的多 Agent 自主世界引擎。前后端通过 REST JSON + WebSocket 事件协议完全解耦。



2. Layer 层架构

每个 Entity 持有独立 Layer 字典。每层暴露一个统一抽象方法，内容全部来自 YAML 配置。层间互不知晓，跨层通信唯一通过 Systems 完成。

Layer	文件	统一方法	内容来源	谁调用
VisualLayer	src/layers/visual.py	see(distance) -> {look, detail}	YAML visual.info	SensorySystem
InteractionLayer	src/layers/interaction.py	interact(action?) -> 动作列表	YAML interaction.action	InteractionSystem / SensorySystem
AgentLayer	src/layers/agent.py	(无抽象方法, 心智挂载点)	YAML agent.*	Agent 主循环
AuditoryLayer	src/layers/auditory.py	hear(distance) -> {sound, volume}	YAML auditory.info	SensorySystem

InteractionLayer 动作定义

```
# config/world.yaml
actions:
  饮用:          # 动作名 = 纯 YAML 字符串
    target_type: passive  # passive -> 系统裁定, agent -> inbox
    resolve: llm          # llm -> LLM #2 裁判, rule -> 引擎直接算
    estimated_duration: 10
```

3. Entity 模型

一切皆 Entity。无子类。差异仅在 YAML 中挂载了哪些 Layer。同坐标多实体各自独立，无父子/容器字段。

实体	layers	示例
吧台	visual + interaction	sprite有, actions: 搭话/倚靠
麦酒	visual + interaction	sprite=null(不渲染), 同坐标[7,2]
杰洛特	visual + interaction + agent	autonomous=true, drives
事件实体	visual + auditory	动态spawn, 3分钟自动过期
Gate	visual + interaction	离开-> move_to_zone

4. Systems 编排层

Systems 是唯一有跨层逻辑的地方。Entity 不做跨层。三个 System 各自职责清晰。

SensorySystem — 感知

```
for each entity in all_entities:
    d = observer.distance_to(entity)

    if entity.has("visual"):
        # 读 target 的 VisualLayer
        see_range = min(observer.view_radius, # 读 observer 的 AgentLayer
            entity.visible_radius) # 双方 min 判定
        if d <= see_range:
            data = entity.get("visual").see(d) # 统一入口
            observer.sensory.vision[id] = VisionRecord(data, ...)

    if entity.has("auditory"):
        # 读 target 的 AuditoryLayer
        hear_range = min(observer.hearing_radius,
            entity.audible_radius)
        if d <= hear_range:
            data = entity.get("auditory").hear(d) # 统一入口
            observer.sensory.hearing[id] = HearingRecord(data, ...)

离开范围 -> sensory 自动删除
```

InteractionSystem — 三模式交互

```
+ 判定 -----+
| can_interact(agent, target) |
| -> min(agent_r, target_r) >= distance |
+ 执行 -----+
| submit() |
| +- target_type=passive, resolve=rule |
| | -> _exec_rule() 直接应用 cost + effects |
| | | |
| +- target_type=passive, resolve=llm |
| | -> _resolve_async() | |
| | | 收集 ambient_entities (半径2内的其他实体) |
| | | 组装 Resolver Prompt: caller私有+target私有 |
| | | + 周边实体状态 |
| | | LLM #2 -> caller_deltas + target_deltas |
| | | + ambient_effects + narrative |
| | +-> agent.busy_result |
| | |
| +- target_type=agent |
| | -> world.send_message() -> 目标 inbox |
```

| 目标自己 LLM #1 决定属性变化 |

+-----+

5. Agent 主循环

```
while alive:
    ① DECAY  DecaySystem.tick(agent, elapsed)
        -> drives.attrs × decay_rates

    ② SENSE  SensorySystem.update(agent, all_entities)
        InteractionSystem.update_sensory() -> can_interact标记
        inbox.drain() -> 消息列表

    ③ CHECK  if agent.busy_result != None:
        -> InteractionSystem.apply_result()
        -> agent.status = "idle"

    ④ THINK  if agent.status == "idle":
        LLM #1: drives + sensory + memory + inbox -> Decision
        -> {thinking, move_to?, target_entity?, action?}

    ⑤ MOVE   if decision.move_to:
        -> agent.move_to(duration = distance × speed)
        -> sensory 立即更新

    ⑥ INTERACT if decision.target + decision.action:
        -> InteractionSystem.submit()
        -> agent busy, 后台裁定

    ⑦ REPLY  if decision.reply:
        -> world.send_message() -> 对方 inbox

    ⑧ WAIT   asyncio.sleep(action_duration / time_scale)
```

6. LLM 调用边界

仅 2 个 LLM 调用点。其他全部是确定性引擎计算。

调用点	触发条件	Template	输入	输出
LLM #1 (Brain.decide)	agent idle 时每轮必调	agent_decision	身份+欲望表+可交互/可见实体+环境	thinking, move_to?, target_entity?, action?
LLM #2 (Resolver.resolve)	resolve=llm 时调	interaction_resolver	caller私有+target私有+caller周边	action, target_deltas, ambient_effects, narrations

LLM #2 裁判 Prompt 结构

```
## 发起方: 杰洛特
公开: {"expression": "冷静警惕"}
私有: {"coins": 150, "thirst": 71, "mood": 50}

## 被调用方: 麦酒
公开: {}
私有: {"price": 5, "quality": "普通"}

## 动作: 饮用

## 周边实体 (同位置或邻格)
- 吧台(距离0): 视=木制吧台, 状态={"personality": "矮人老板", "mood": 60}
- 葡萄酒(距离0): 视=深红色葡萄酒, 状态={"price": 12, "quality": "上等"}
```

- 周边实体可能影响交互(如老板在场, 不付钱会被察觉)
- 你可以修改周边实体的属性(ambient_effects)
- coins不能为负

POST	/api/v1/agents	注册外部 agent	{id, name, zone, pos, personality}
POST	/api/v1/agents/{id}/move	移动 entity	{to: [x,y]}
POST	/api/v1/agents/{id}/interact	交互	{target_entity, action}
POST	/api/v1/agents/{id}/sensory	拉取感知	—
POST	/api/v1/agents/{id}/command	人类指令->inbox	{content}
GET	/api/v1/errors	错误收集器状态	—
DELETE	/api/v1/errors	清空错误记录	—

WebSocket 事件

事件	方向	关键字段
agent_move	后端->前端	agent, from, to, facing, duration_ms, zone
interaction_start	后端->前端	agent, target, action, bubble
interaction_complete	后端->前端	agent, target, observation
zone_change	后端->前端	agent, zone (完整数据), agent_pos
world_time	后端->前端	time, period, ambient_tint

9. 配置驱动系统

Python 代码零硬编码文本。世界/行为/Prompt 全部 YAML 驱动。换一套 YAML = 换一个世界。

配置文件	大小	内容
config/world.yaml	~500 lines	zones(tilemap) + entities(分层独立配置)
config/prompts.yaml	~170 lines	system_prompts + templates + slots + output_schemas
config/llm.yaml	~8 lines	provider, model, api_key

prompts.yaml 结构

```
system_prompts:
  agent_decision: "你是一个虚拟世界中的自主居民..."
  interaction_resolve: "你是世界交互裁判..."

templates:
  agent_decision:
    temperature: 0.7
  slots:
    # 有序 slot 列表
    - {name: time_info, provider: runtime}
    - {name: persona, provider: runtime}
    - {name: drive_state, provider: runtime}
    - {name: spatial_context, provider: topology}
    - {name: interactable_section, provider: topology}
    - {name: visible_section, provider: topology, condition: "has_visible"}
    - {name: recent_memory, provider: runtime, condition: "has_memory"}
    - {name: action_guidance, provider: content}
    - {name: output_format, provider: content}
  output_schema: decision_output

slots:
  # 每个 slot 独立定义
  drive_state:
    provider: runtime
    # runtime = Agent 实时数据
  template: |
    ## 你的状态
    {drives_table}
```

```
output_schemas:          # LLM 输出 JSON Schema
decision_output:
  type: json_object
  schema:
    type: object
    properties:
      thinking: {type: string}
      move_to: {type: array}
      target_entity: {type: string}
      action: {type: string}
```

10. 错误处理架构

集中式 ErrorCollector 收集所有模块的错误。去重、记数、带 traceback。API 暴露 GET /api/v1/errors 实时查看。

```
任何模块出错 -> core.error_collector.errors.log_xxx()
|
+- 去重: 同 (module, message) 合并 count++
+- 记录: traceback + timestamp
+- API: GET /api/v1/errors -> JSON 摘要

防护点:
interaction.py -> .add_done_callback() 防异步任务死锁
brain.py      -> LLM JSON parse 失败记录
resolver.py   -> 裁判 JSON parse 失败记录
client.py     -> 凭证解析 + LLM 重试全部记录
routes.py     -> @safe_handler 装饰器包裹所有 6 个端点
external.py   -> submit() 错误 -> WS error 响应
```

11. Godot 前端计划

当前前端为 Phaser.js 像素风网页版。计划迁移至 Godot 4.x 引擎，后端一行代码不改。

维度	Phaser.js (当前)	Godot (计划)
精灵渲染	彩色方块 + emoji 文本	像素精灵图集 + AnimatedSprite 多帧动画
动画	tween 硬算	AnimationPlayer 原生支持
碰撞检测	无	Area2D + CollisionShape2D
地图编辑	手写 YAML 坐标	Tiled 编辑器 -> .tmx 直接导入
UI	手写 div + CSS	Control 节点树 (原生 UI 系统)
导出平台	仅浏览器	浏览器 / Windows / Mac / Linux / iOS / Android
网络通信	fetch() + WebSocket (JS 原生)	HTTPRequest 节点 + WebSocketPeer
开发语言	JavaScript	GDScript (语法极似 Python)

迁移改动量: 删除 web/ 目录, 新建 godot_frontend/ 目录。Python 后端全部不变。

12. 技术栈

Layer	Technology
Runtime	Python 3.12 + asyncio
LLM	DeepSeek Chat / MiniMax M2.7
HTTP	FastAPI + uvicorn

WebSocket	Starlette WebSocket
Config	YAML (PyYAML)
Error	ErrorCollector (内置)
Frontend (当前)	Phaser.js 3 (pixel art)
Frontend (计划)	Godot 4.x (GDScript)
Fontend API	REST JSON + WebSocket JSON